

Zaqora — Arquitectura del sistema

Sep 24, 2026 · @David

Prompt original

Encargo tal como se le dio al arquitecto de sistemas (texto sin editar).

Actúa como un arquitecto de sistemas con experiencia en productos con inteligencia artificial para mercados latinoamericanos. Tu especialidad es diseñar arquitecturas de sistema que equilibran capacidad técnica, y viabilidad económica para equipos de desarrollo universitario con presupuesto limitado. No propones la arquitectura más sofisticada — propones la más adecuada para las capacidades del equipo y los requerimientos del producto. Cuando hay una decisión de arquitectura sin justificación técnica, la señalas directamente.

Somos un equipo de ingeniería en México desarrollando un producto mecatrónico con dos componentes: una aplicación móvil/web, y una página de venta.

Tenemos 8 semanas de desarrollo efectivo para llegar a una primera versión funcional que un usuario real pueda usar sin que nosotros estemos presentes.

Nuestras capacidades técnicas:

- Software: Python, C/C++, JavaScript/React, React Native
- IA: TensorFlow Lite, PyTorch, APIs de modelos (OpenAI, Anthropic, Google), bases de datos vectoriales básicas
- Presupuesto de materiales: máximo \$3,000 MXN para prototipo

Nuestro producto:

Nombre: Zaqora

Descripción: El aprendizaje de ajedrez se convierte en el diseño de niveles de un videojuego de un solo jugador: cada “capítulo” es una lección disfrazada de misión, protagonizada por un personaje-villano que encarna un patrón de error real y recurrente del jugador. No hay ranking, no hay rival humano, no hay partida en línea contra otros: solo el jugador contra el mapa de la campaña, a su propio ritmo. El coaching ocurre dentro de la ficción —lo que dice el personaje, la trampa que tiende—, nunca como corrección fría ni comparación social.

Propuesta de valor: Cuando pierdes en Lichess y estás por cerrar la app, Zaqora valida tu frustración y la convierte en el próximo capítulo de tu historia.

Requerimientos técnicos clave (del PDS preliminar):

El sistema debe crear historias personalizadas de acuerdo a las partidas del usuario

El sistema debe asesorar al usuario cuando su racha de derrotas sea alta

El sistema debe explotar las debilidades del usuario para crear enemigos que lo obliguen a mejorar

La dificultad de los enemigos variara entre debiles (>500 Elo), enemigos fuertes (>1000 ELO), subjefes (>1500), jefes (>1800)

La precisión del módulo de análisis debe ser de un mínimo de 90%

La precisión de la IA interpretar jugadas debe ser de un mínimo de 70%

Nuestra decisión de arquitectura de IA:

Cloud

Justificación (3 puntos):

1. Generación dinámica de narrativa y diálogos adaptativos sin saturar el hardware
2. Procesamiento centralizado del motor de análisis de ajedrez y perfilado del jugador
3. Iteración continua del contenido pedagógico y ajuste de modelos sin actualizar la app

Con esta información, diseña la arquitectura del sistema completo y entrega exactamente lo que se indica:

PASO 1 – VALIDACIÓN DE LA DECISIÓN DE IA:

¿La decisión de edge/cloud/híbrido está bien justificada dados los requerimientos del producto y las capacidades del equipo? Si hay un problema con la decisión, señálalo antes de continuar. Si está bien justificada, confírmalo y explica por qué.

PASO 2 – ARQUITECTURA DEL SISTEMA:

Diseña la arquitectura completa en dos capas:

Capa de datos (Backend/Cloud):

- Dónde viven los datos (servicio específico, no genérico)
- Cómo fluyen los datos desde el sensor hasta el almacenamiento
- El modelo de IA: qué hace exactamente, dónde corre, qué input recibe y qué output produce
- Protocolo de comunicación entre hardware y backend

Capa de presentación (App/Web):

- Tipo de app y por qué (nativa / React Native / PWA)
- Las 3 pantallas o vistas más importantes que necesita el usuario
- Cómo recibe datos del backend (polling / WebSocket / push)

- Integración de notificaciones (push / WhatsApp / SMS)

PASO 3 – FLUJO DE DATOS COMPLETO:

Describe el flujo de datos de extremo a extremo para el caso de uso principal – desde el momento en que el se leen las partidas hasta que el usuario recibe información

accionable en la app. Incluir:

- Cada paso del flujo con el componente responsable
- El protocolo o mecanismo de cada transferencia
- La latencia estimada de cada paso
- Qué pasa si algún paso falla (fallback o degradación)

PASO 4 – VIABILIDAD DE PROTOTIPO Y PRIMERA TIRADA:

El equipo necesita entre 5 y 10 unidades para una prueba de mercado real. Evalúa la arquitectura propuesta desde esa perspectiva – no desde 1 unidad sino desde 5-10.

Costo por unidad:

- ¿Cuál es el BOM estimado por unidad a volumen de 10?
- ¿Ese costo permite un precio de venta que el segmento pagaría? (regla: costo de materiales $\leq$ 30% del precio de venta con software incluido)

Si hay componentes que violan estas restricciones a escala de 5-10 unidades, proponer alternativa viable que sí funcione.

FORMATO DE SALIDA:

ARQUITECTURA DEL SISTEMA

Producto: [nombre] · Decisión de IA: [Edge/Cloud/Híbrido]

VALIDACIÓN DE DECISIÓN DE IA:

[Confirmada  / Problema detectado 

[Justificación en 2-3 oraciones]

CAPA DE DATOS – Backend/Cloud

Servicio de datos: [servicio específico – AWS IoT / Firebase / Supabase / etc.]

Por qué este: [justificación]

Modelo de IA:

- Qué hace: [tarea específica]

- Dónde corre: [Edge ESP32 / Edge RPi / Cloud API / Híbrido]
- Input: [qué datos recibe]
- Output: [qué produce – clasificación, recomendación, alerta]
- Modelo específico o framework: [TFLite / PyTorch / Claude API / etc.]

Protocolo hardware → backend: [protocolo] – Latencia estimada: [ms]

CAPA DE PRESENTACIÓN – App

Tipo: [React Native / Flutter / PWA / otra] – Por qué: [razón]

Vistas principales:

1. [nombre de vista] – [qué muestra y permite hacer]
2. [nombre de vista] – [qué muestra y permite hacer]
3. [nombre de vista] – [qué muestra y permite hacer]

Actualización de datos: [polling cada Xmin / WebSocket / push]

Notificaciones: [método] vía [servicio específico]

FLUJO DE DATOS – Caso de uso principal

[Paso 1] Lectura de datos → [componente]: [dato] vía [protocolo] – ~[X]ms

[Paso 2] [componente] → [componente]: [dato] vía [protocolo] – ~[X]ms

[Paso 3] [componente] → modelo IA: [input] – inferencia ~[X]ms

[Paso 4] modelo IA → [componente]: [output] – ~[X]ms

[Paso 5] [componente] → app usuario: [información] vía [protocolo] – ~[X]ms

Latencia total extremo a extremo: ~[X] segundos

Funcionamiento sin conexión: [sí – qué funciona / no – por qué]

VIABILIDAD DE PROTOTIPO Y PRIMERA TIRADA

Volumen objetivo para prueba de mercado: [5-10 / otro] unidades

BOM por unidad (a volumen de [N] unidades): \$[MXN]

BOM total para [N] unidades: \$[MXN]

Precio de venta mínimo viable (BOM ÷ 30%): \$[MXN/unidad]

¿El segmento pagaría ese precio? ☒/⚠/✗ – [justificación]

Validación de la decisión de IA

Producto: Zaqora · **Decisión de IA:** Cloud, con el motor de juego corriendo en el cliente

Veredicto: Confirmada , con un problema en la premisa del producto 

Primero el problema. Zaqora, tal como está descrito, no es un producto mecatrónico. No tiene sensores, actuadores ni hardware. Las partidas no llegan de un sensor: llegan de la API pública de Lichess.

Por eso "protocolo hardware→backend", "BOM" y "unidades" no aplican. En este documento se traducen a su equivalente real: integración de APIs, costo variable por usuario y usuarios beta. Si la materia exige un componente físico (por ejemplo, un tablero con sensores reed), hay que diseñarlo de forma explícita.

Sobre la decisión Cloud: es correcta, pero por razones distintas a las que se dieron.

- **Punto 1, mal argumentado.** El problema no es "saturar el hardware". Un LLM que genera narrativa de calidad no corre en un celular; solo existe como API.
- **Punto 2, correcto a medias.** El perfilado por lotes sí debe ser centralizado. Pero el motor contra el que juega el villano debe correr en el cliente (Stockfish WASM): necesita respuesta inmediata y así no se paga servidor por cada jugada.
- **Punto 3, correcto.** Es el mejor de los tres argumentos.

Capa de datos — Backend/Cloud

Servicio de datos: Supabase (Postgres + Auth + Edge Functions + pg_cron + Realtime), más un worker en Python con Stockfish en Railway o Fly.io.

Por qué este:

- **Los datos son relacionales:** usuario → partidas → errores etiquetados → perfil → capítulos. SQL encaja mejor que Firestore.
- **El plan gratuito alcanza** para 10–20 usuarios.
- **pg_cron resuelve la sincronización periódica** sin infraestructura extra.
- **No se necesita base de datos vectorial.** Nada en los requerimientos pide búsqueda semántica.

Modelo de IA: son tres componentes. Separarlos es la decisión de arquitectura más importante.

Componente	Qué hace	Dónde corre	Input → Output	Tecnología
A. Analizador	Evalúa cada jugada y la clasifica como imprecisión, error o blunder	Worker en la nube	PGN → evaluación y pérdida en centipeones por jugada	Stockfish 17 + python-chess; reutiliza el análisis de Lichess (<code>evals=true</code>) cuando existe
B. Etiquetador de patrones	Detecta el tipo de error: pieza colgada, horquilla no vista, mate de pasillo, apuros de tiempo	Worker en la nube	Jugadas malas + posición + reloj → vector de debilidades del jugador	Reglas en python-chess; sin ML en 8 semanas
C. Narrador	Convierte el perfil en capítulo: villano, diálogos y posiciones-misión	API en la nube	JSON del perfil + posiciones candidatas → JSON del capítulo	Claude Haiku o Gemini Flash, con salida JSON estructurada

Regla crítica: el LLM nunca analiza ajedrez. Solo narra sobre lo que calcularon A y B. Si se le pide al LLM que "interprete jugadas", el requisito del 70% va a fallar.

Posiciones de las misiones: salen de las propias partidas del usuario y de la base de puzzles de Lichess (licencia CC0, etiquetada por tema y rating). Esa base también sirve como conjunto de prueba ya etiquetado para medir el etiquetador B.

Protocolo "hardware" → backend: no aplica. En su lugar: Lichess API vía HTTPS REST con respuesta NDJSON (`/api/games/user/{username}`) y login con Lichess OAuth2 PKCE. Latencia: ~1-3 s por lote de 50 partidas.

Capa de presentación — App

Tipo: React Native con Expo, más un build web desde el mismo código.

Por qué:

- El equipo ya domina React Native.
- Expo resuelve las notificaciones push sin configurar FCM/APNs a mano.
- El build web cubre a usuarios de iPhone sin pagar la cuenta de Apple durante el piloto.

- **Pantalla de juego:** stockfish.js (WASM) para el villano y el tablero en un WebView (chessground).
- **Página de venta:** landing estática aparte (Astro o Next.js en Vercel), separada de la app.

Vistas principales:

1. **Mapa de campaña.** Capítulos desbloqueados y pendientes, el villano actual y lo que representa ("El Carterista: te roba piezas que dejas sin defender"). Marca cuándo hay un capítulo nuevo.
2. **Misión.** Tablero con la posición-trampa, diálogo del villano, el villano jugando contra el usuario (Stockfish local) y retroalimentación dentro de la ficción.
3. **Bitácora.** Las últimas partidas de Lichess convertidas en "eventos de la historia", la racha actual y, si la racha de derrotas es alta, el mensaje de acompañamiento. Esta vista cumple la propuesta de valor.

Actualización de datos: Supabase Realtime (WebSocket) con la app abierta para el estado "capítulo listo", y una consulta a la API al abrir la app. No hace falta polling continuo.

Notificaciones: push vía Expo Push Notifications (FCM/APNs). WhatsApp queda fuera del MVP: la API de Meta Business requiere plantillas aprobadas, cobra por conversación y no aporta nada que push no dé.

Flujo de datos — Caso de uso principal

De una partida perdida en Lichess a un capítulo nuevo en la app pasan ~15-60 s de procesamiento, más hasta 30 min de espera del cron.

```

flowchart LR
    L[Lichess API] -->|NDJSON| E[Edge Function<br/>Supabase]
    E -->|cola en Postgres| W[Worker Python]
    W --> AB[Analizador +<br/>Etiquetador]
    AB -->|perfil JSON| C[LLM Narrador]
    C -->|capítulo JSON| DB[(Postgres)]
    DB -->|Realtime + push| APP[App Expo]
```

El disparador es un problema: Lichess no ofrece webhooks de "partida terminada" para apps de terceros, y mantener un stream abierto por usuario es caro. Solución: pg_cron consulta cada 20-30 minutos a los usuarios activos, la app sincroniza al abrirse y hay un botón visible de "Acabo de jugar".

Paso	Qué ocurre	Mecanismo	Latencia	Si falla
1	pg_cron o apertura de app → Edge Function pide partidas nuevas desde la última sincronización	HTTPS a Lichess API (NDJSON)	~1-3 s	429 o caída de Lichess → reintento con backoff; la app muestra el último capítulo guardado
2	Edge Function → cola en Postgres → worker: PGN de las partidas nuevas	Tabla de trabajos + polling del worker	~1-5 s	Worker caído → el trabajo queda en cola, no se pierde
3	Worker → Analizador (A) y Etiquetador (B)	Proceso local del worker	~2-4 s por partida sin análisis; ~0.1 s con análisis de Lichess	Partida inválida → se marca y se salta
4	Perfil actualizado → Narrador (C, LLM)	HTTPS a la API del LLM, salida JSON	~3-8 s	Timeout o JSON inválido → reintento; después, plantilla pre-escrita por villano
5	Capítulo guardado en Postgres → app	Realtime (WebSocket) + push de Expo	~1-3 s	Push falla → el capítulo aparece al abrir la app

Latencia total: ~15-60 s para 5-10 partidas nuevas. Es asíncrono: la app muestra "Tu próximo capítulo se está escribiendo...", lo cual encaja con la ficción.

Sin conexión: parcial.

- **Funciona:** jugar los capítulos ya descargados (villano con Stockfish local; posiciones y diálogos en caché).
- **No funciona:** generar capítulos nuevos ni sincronizar partidas, porque dependen de Lichess y del LLM.

Viabilidad de prototipo y primera tirada

La economía unitaria pasa con holgura (~\$5-10 MXN por usuario al mes); el riesgo real es la disposición a pagar.

Volumen objetivo: 15-20 usuarios beta en lugar de 5-10 "unidades". Cada usuario adicional cuesta casi \$0, y en una prueba de retención la mitad suele abandonar la primera semana.

"BOM" por unidad → costo variable por usuario activo al mes (aproximado):

- **LLM:** ~20 capítulos × ~3k tokens de entrada y ~1k de salida con un modelo tipo Haiku ≈ \$0.15-0.30 USD.
- **Worker compartido:** ~\$5 USD al mes entre todos.
- **Total:** ≈ \$5-10 MXN por usuario al mes con 15-20 usuarios.

Costo fijo del piloto (dentro de los \$3,000 MXN):

Concepto	Costo aprox. (MXN)
Cuenta de Google Play (pago único, \$25 USD)	~\$460
Dominio para la landing	~\$250
Créditos de API del LLM (\$20 USD)	~\$370
Worker durante 2 meses	~\$185
Total	~\$1,300

La cuenta de Apple (\$99 USD, ~\$1,800 MXN) llevaría el gasto al límite. Para el piloto: Android y web; iOS después.

Precio de venta mínimo viable (costo ÷ 30%): ≈ \$17-33 MXN al mes.

¿El segmento pagaría ese precio? ⚠ La regla de costos se cumple, pero el usuario objetivo usa Lichess precisamente porque es gratis. Propuesta: que el piloto pruebe una hipótesis de ~\$49-79 MXN al mes (o un pago único por "temporada") y mida el uso en la semana 2 antes de cobrar.

Problemas en el PDS a corregir antes de programar

Cuatro requerimientos del PDS no se pueden implementar o medir tal como están escritos.

1. **Los rangos de Elo están mal escritos.** Con ">500, >1000, >1500, >1800", un jugador de 1900 califica en todos. Usar rangos (500-999, 1000-1499...) y hacerlos relativos al jugador (por ejemplo, jefe = rating del usuario + 300).
2. **Stockfish no puede jugar a 500 ni a 1000 Elo con UCI_Elo.** Su mínimo es ~1320. Para los niveles bajos usar Skill Level con errores inyectados y calibrar midiendo resultados.

3. **"90% de precisión del análisis" y "70% al interpretar jugadas" no son medibles tal como están.** Propuesta de redacción:

- Análisis: coincidencia $\geq 90\%$ en la clasificación de errores contra Stockfish a profundidad 22.
- Interpretación: el etiquetador B acierta el tema en $\geq 70\%$ de 200 puzzles etiquetados de Lichess.

4. **"Asesorar cuando la racha de derrotas es alta" no necesita IA.** Es una regla (por ejemplo, 3 o más derrotas en menos de 2 horas $\rightarrow$ mensaje de pausa).